

# Mark AI — What I Built (Current Role)

 **Role:** Software Engineer (Full-time) · **Since:** June 2026 · **GitHub org:** Adneuron

**Mark AI** is an AI-native **DOOH** (digital out-of-home) advertising platform. Advertisers book ad slots that play on physical Android signage screens in cafés and venues — and Mark AI owns and operates those screens itself. I own the stack end to end: the backend that is the fleet's source of truth, the Kotlin player and updater apps that run on every screen, the operator CMS on web and phone, the advertiser platform, and the AI / creative / 3D layers on top.

## Contribution at a glance

| Metric                                                     | Value                                                                     |
|------------------------------------------------------------|---------------------------------------------------------------------------|
| Pull requests authored (current platform)                  | <b>319</b> (313 merged)                                                   |
| Commits on the current platform                            | <b>482</b> since 15 Jun 2026 (as of 2 Oct 2026)                           |
| Authorship                                                 | <b>Sole engineer</b> on all 12 repositories of the current platform       |
| CMS-Back — signage backend                                 | 162 commits · <b>131 PRs</b> · Python/FastAPI                             |
| CMS-Web — operator CMS                                     | 116 commits · <b>97 PRs</b> · Next.js/TypeScript                          |
| APK-CMS — Android player                                   | 51 commits · <b>38 PRs</b> · Kotlin · 165 unit tests                      |
| Markai-Updater — Android updater app                       | 13 commits · <b>4 PRs</b> · Kotlin                                        |
| CMS-Mobile — CMS phone app                                 | 27 commits · <b>10 PRs</b> · Kotlin Compose Multiplatform (Android + iOS) |
| Mark-Ai-front-new / Mark_Ai-back_new — advertiser platform | 57 commits · <b>33 PRs</b> · Next.js 15 + FastAPI                         |
| Mark-ai-studio / API-studio — Mark AI Studio               | 34 commits · Next.js + FastAPI                                            |
| MarkAi-dev · MVP-AR · AR-model · Mark-ai-secrets           | GCP cost dashboard · AR showroom · 3D studio · infrastructure-as-code     |
| Backend modules                                            | 27 domains over <b>84 Alembic migrations</b>                              |

## The architecture I own



## What I built — by theme

### 1 · The signage backend & the device contract

- Built **CMS-Back** from the first commit: a domain-driven **FastAPI + PostgreSQL** service, now 27 modules (`screens`, `bookings`, `loops`, `schedule`, `playback`, `content`, `screenshots`, `delivery`, `reports`, `updater`, `apk`, `mobile_app`, `api_keys`, `cms_users`, `ai`, `whatsapp`, `gmail`, `creative_fit`, `health` and more) across **84 Alembic migrations**.
- **Removed the WebSocket** and folded the live channel into the heartbeat — `register` · `heartbeat` · `schedule` · `proof`. Each heartbeat carries `reload_needed`, `now_playing` and pending screenshot jobs, so a CMS edit reaches a screen within one interval with **no held sockets**. That single decision is what makes the fleet horizontally scalable.
- **Paged the fleet in SQL** instead of shipping every row, and capped Cloud Run `max-instances` so the service can never exhaust the Cloud SQL connection pool.
- *Representative work*: CMS-Back — "update the cms and remove the websocket" · "add the functionality of POP and make batching" · #13 page the fleet in SQL · #14 cap max-instances to protect Cloud SQL

### 2 · The Android signage player (Kotlin)

- Built the player APK that runs fullscreen 24/7 on physical Android boxes — self-registers on a permanent device UID, holds the calendar and starts each ad on its own boundary, survives reboot, and runs on AOSP with **no Google Play Services**.
- **Never goes dark**: when a screen's token expires it re-registers itself instead of silently dropping off the fleet.
- **Operating-hours behaviour** — outside its window the screen shows black and drops to a slow keep-alive heartbeat rather than going silent, so an hours change still reaches a closed screen on its next check-in.
- **Location on hard devices** — auto-detection via the platform `LocationManager` + built-in `Geocoder` (no API key), an operator-triggered re-scan delivered over the heartbeat, and a fallback path for a screen that takes its internet from an **Ethernet cable** with no GPS or WiFi to geolocate from.
- *Representative PRs*: APK-CMS #5/#6 operating hours + slow off-hours keep-alive · #7 fresh GPS fix at capture · #8 tag plays with booking id · #9 enrollment-key gate · #11 enrol from a baked-in key + release pipeline · #12 re-register on token expiry · #13 locate an Ethernet-only screen

### 3 · Proof-of-play & screenshot evidence (the billing trail)

Proof-of-play is what advertisers are billed against, so most of the hard correctness work lives here.

- **Attribution** — every play is tagged with its `booking_id` rather than just screen + media, so two campaigns sharing a creative can't steal each other's counts.
- **Milestone screenshots armed on the playable timeline**, not a UTC clock hour and not the raw campaign span — a campaign that can only play inside a screen's open hours gets its milestones where the ad actually runs, and a cancelled campaign marks them *missed* instead of pending forever.
- **Capture correctness** — fire on the ad's render callback instead of a one-second timer; read video frames from the `SurfaceView` rather than the window; never capture a half-rendered multi-zone layout; never photograph the same milestone twice; one upload worker per milestone with orphaned frames swept up.

- **Evidence hardening** — a GPS-camera-style overlay burned into each milestone shot, backed by a fresh location fix taken at capture time.
- **On-demand "Capture now"** — an operator can demand a live frame from any screen, gated on a real play, with 24-hour retention, per-frame deletion and sane download naming.
- *Representative PRs:* CMS-Back #17 GPS overlay · #18 attribute plays by booking\_id · #31/#32/#33 on-demand capture · #39 retention + download naming · CMS-Web #23 "Capture now" · #26 capture downloads & table view · APK-CMS #7 fresh GPS fix

#### 4 · Scheduling, operating hours & time

- **Per-screen operating hours** (default 09:00–17:00) enforced end to end — the backend rejects bookings outside a screen's window, the CMS shows an *Off-hours* tag and tells the operator when an hours change will actually land on a closed screen, and the player gates playback locally.
- **All wall-clock and calendar logic interpreted in IST** as a core decision — a bug where the AI booked into the wrong year traced straight back to timezone handling.
- **A booked campaign's schedule is immutable** — only its name can change once it's live; editing is a separate, explicit flow.
- **An honest lifecycle** — three booking statuses with a terminal Cancel and no Delete button, and a campaign that flips to *Completed* the moment it can no longer play.
- *Representative PRs:* CMS-Back #16 operating hours · #12 calendar schedule · #11 schedule lock · CMS-Web #11/#12 hours in the CMS + off-hours tag · #10 completed state · #9 three statuses & terminal cancel · IST core commits

#### 5 · The AI co-pilot (differentiator)

- A conversational agent inside the CMS that turns plain English into a real campaign booking — tool-calling over the live domain, a draft → confirm loop, and **two layers of guardrails**.
- **It never re-asks** — a deterministic slot-capture backstop fills the booking draft before the main agent runs, so a detail the operator already gave is never requested twice.
- **It knows what day it is** — a current-datetime tool plus per-turn IST context, after bookings were landing in the wrong year.
- **Campaign Q&A and proof-of-play analysis** — lookup by campaign name, date-range listing, operating-hours-aware validation at booking time, and strict gap/health analysis so it can answer "did this actually play?" without guessing.
- **Scoped per screenowner** — read tools and booking executors both respect who is asking.
- **Evaluated with an actor/verifier agent harness** driving real chat sessions against a seeded database, written up as a test report — not unit tests.
- *Representative PRs:* CMS-Back #21 per-screenowner co-pilot scoping · #34 campaign Q&A, PoP gap analysis & operating-hours awareness · #3/#5 current-date tool & per-turn context · guardrail commits

#### 6 · Location intelligence & fuzzy matching

"Book me the screens near Connaught Place" only works if the system can survive a misspelling and rank by real distance. So I built the matching layer myself:

- A **typo-tolerant text scorer** to replace substring matching, scoring on **word coverage** rather than a weighted average — then **ported the same scorer into the CMS frontend and guarded both copies with a shared golden table** so search behaves identically on either side.

- **Haversine distance, a padded fleet bounding box and forward geocoding**, so screens rank by text match first and then by true distance, with the agent taught explicit distance bands.
- **Locality misspelling correction** against the actual fleet's cities, capped result counts, and an explicit "ask when unsure" path instead of guessing.
- Typo-tolerant search reused across screens, the media picker and user assignment.
- *Representative PRs*: CMS-Back #8 AI location resolution (haversine, bbox, forward\_geocode, distance bands, misspelling correction) · CMS-Web #6 fuzzy scorer ported + golden table, resolved place & per-screen distance on the chat card

## 7 · Enrollment & APK distribution (zero-touch)

- A **per-screenowner enrollment key** with self-service regeneration in the CMS, and a first-launch gate on the player.
- **Per-owner pre-keyed APKs** — an owner downloads a build with their enrollment key already baked in, so installing a screen needs no typing and no technician. Backed by a release pipeline and a Player-app section in the CMS.
- **Fleet version reporting** — every screen reports which build it is running, and a box can ask the backend whether enrollment is even required before it registers.
- *Representative PRs*: CMS-Back #19 per-screenowner enrollment key · #35 enrollment-policy pre-check · #37 per-owner APKs + release pipeline + fleet versions · #38 claim a build by key · CMS-Web #14/#24/#25 · APK-CMS #9/#10/#11

## 8 · Roles, permissions & the partner API

- Designed the role model — **superadmin** (full control, assigns screens) and **screenowner** (scoped to their own screens) — then drove **screenowner parity** across every surface: solutions, content, bookings, AI, enrollment keys, navigation.
- Moved operator identity to **email** and shipped forgot/reset password over a single-use emailed link, plus a transactional mail path on the company domain.
- Published the `/api/ext/*` **partner API** — API-key auth, per-key booking attribution with an external advertiser reference, one-call screen detail with slot occupancy, per-booking play logs, and pause/resume/cancel semantics for partner bookings.
- *Representative PRs*: CMS-Back #20 screenowner parity · #22 ext-API attribution · #23 screen detail + play log · #24 partner pause/resume/cancel · #25 operator identity · #28/#29 password reset · CMS-Web #15/#16/#19/#20/#21

## 9 · The advertiser platform (Next.js 15 + FastAPI)

- Built the public product — marketing site with a **3D city hero** and an AR hologram section, a unified **email-first auth** flow, and a signed-in advertiser experience: browse screens, screen detail, booking wizard, dashboard and proof-of-play reports.
- A **live slot map** on screen detail that shows a loop as taken / yours / open, tells the viewer how many taken slots are their own, and previews a booking slot before it's confirmed.
- A **screenowner console** — full proof of play, booking detail, a needs-attention panel, and user management.
- Its backend (`Mark_Ai-back_new`) is a second FastAPI/Postgres service that speaks to the CMS as a client: a public screen catalog, a content library that **dual-writes to its own GCS and to the CMS**,

and bookings mapped through `external_ref` — which is exactly how the public product plugs into the same fleet without a second source of truth.

- *Representative PRs*: front-new #1 post-login design revamp · #2 post-login refinements + live slot map · #4/#5 console proof-of-play · #9/#10 console user management & capture · #12 screen sales narrative, console scheduling and Gmail · #13 console proof of play with the CMS's five filters · #15 container image for Cloud Run · back\_new platform domains, campaign passthroughs, seeded dev login

## 10 · Booking approval workflow

- A four-part series across three repos: bookings now **wait for the screen owner's decision** before they play. Operators are created and signed in by email, the approvals console lives on the advertiser platform, and the proof of play is attributed to the operator who approved it.
- *Representative PRs*: CMS-Back #25 (1/4) · CMS-Web #18 (2/4) · back\_new #3 (3/4) & #4 approver attribution · front-new #3 (4/4) approvals console

## 11 · Payments, reports & exports

- **Razorpay** integrated into the current platform — on the signage backend and on the advertiser platform, alongside a profile section and screenowner dashboard build-out.
- **Reporting** — a downloadable campaign **PDF report**, a full proof-of-play **Excel export**, milestone frame downloads, an interactive plays-trend chart, and a needs-attention panel that names the screens that should be on but aren't.
- *Representative PRs*: CMS-Back #30 Razorpay · #41 campaign PDF, milestone downloads, PoP Excel export · CMS-Web #17 plays trend chart · #19 needs-attention panel · #28 report downloads · front-new #8 / back\_new #6 payments

## 12 · Creative Fit, 3D & AR

- **Creative Fit** — refits a finished advertisement to a screen's real resolution: a Gemini-generated layout blueprint, **LaMa** inpainting to erase original elements, and a Pillow relay layout pass, with a workbench UI in the CMS. Output stays native/4K because these play on physical premium screens.
- **3D studio** (`AR-model`) — image/text → 3D asset generation with a self-hosted **Hunyuan3D** on a Cloud Run **L4 GPU** service.
- **AR showroom MVP** (`MVP-AR`) — view-in-AR with QR entry and payment, so a client can preview an advertisement in a real environment before the campaign goes live.
- *Representative PRs*: CMS-Back #40 Creative Fit · CMS-Web #27 Creative Fit workbench · MVP-AR initial build · AR-model + a Hunyuan3D-2.1 fork

## 13 · Cloud & delivery

- The platform runs on **Google Cloud Run** in front of **Cloud SQL**, with a deployment pipeline per service and Cloud Build image builds.
- **Infrastructure as code** — a dedicated `Mark-ai-secrets` repo whose CI applies `services/` to Cloud Run on push to `main`, with a read-only diff step that can't fail the apply job.
- Worked inside real GCP org-policy constraints: **Workload Identity Federation** instead of service-account keys, and **signed URLs** instead of public buckets — applied to the dashboard, the AI tools and proof-of-play screenshots.

- **Creative uploads go direct to GCS** with a real progress bar; the proxied upload endpoint through Cloud Run was deleted rather than left to rot.
- **Performance pass** — every list endpoint paginated and searched server-side, a page's GCS URLs signed concurrently, and the shared signing credentials locked against a refresh race.
- *Representative PRs*: CMS-Back #7 direct GCS upload (+ delete the proxy) · #42 paginate and search every list endpoint server-side · #43 sign GCS URLs concurrently · #69 lock signing credentials against a refresh race · signed-URL credentials commits · deployment-pipeline commits · Mark-ai-secrets CI

## 14 · The Updater app — a second APK that keeps the Player updated (Kotlin)

Screens sit in other people's cafés and shops. Updating the Player used to mean reaching each box by hand or over adb. I designed and built a second app that does it without anyone touching the box, so a box now needs exactly two APKs: the Player and the Updater.

- **Private network built in** — the Updater runs the Tailscale engine (`tailscaled`, built from source for Android, userspace networking) as its own child process and joins our **Headscale** server with a join key the CMS stamps into its download. The box gets a private-network IP with no VPN app and no prompt, and keeps that identity across restarts, reboots and self-updates.
- **Check-in** — every 5 minutes (every minute while an install is in progress) it reports its own version, the Player's version, tunnel state and IP, Device Owner state and recent Player crashes.
- **Silent install** — as **Android Device Owner** it installs the Player with no on-screen prompt. Before installing it verifies the sha256, the package name and the version code, then relaunches the Player and reports the result. It updates itself the same way.
- **Health window and automatic rollback** — after an install the backend watches the screen (heartbeats, foreground ratio, reported crashes). An unhealthy build fails the job and triggers a rollback: the last good release is rebuilt under a higher version code through a GitHub `workflow_dispatch` and reinstalled only on the screens it failed on.
- **Two apps, one signature** — the Player exposes the screen login, device and screen ids and recent crash exits through a content provider guarded by a **signature permission**, so only an app signed with the same key can read it. The release pipeline fails if the two certificates differ.
- **Surviving the real world** — starts at boot, runs as a foreground service with a watchdog, restarts a tunnel that stops, retries an install the box lost power during, and repairs the system clock from our server when HTTPS fails because the clock is wrong.
- **Auto-start lock** — a CMS command turns the box's auto-start lock on and off, and a Device Owner release takes every lock with it so a box can be handed back clean.
- **Both update paths kept** — a screen whose Updater checked in within 15 minutes is updated through the Updater; any other screen keeps the older adb-over-Tailscale agent path, unchanged.
- **Backend and CMS side** — `apk_releases` per app kind (Player / Updater) and per Android floor with its own "latest"; check-in, job-status and artifact endpoints on the screen token; a cron sweep; superadmin rollback, release-Device-Owner and update actions; an Updater status card, Download Updater, and per-screen or bulk update in the CMS. "Update now" stays locked until the *screen* confirms the new version.
- **Release pipeline** — a git tag builds the tunnel engine from source, builds and signs the APK with the Player's keystore, verifies the certificate digest, uploads to GCS under the version code, registers the release with the CMS and publishes a GitHub Release, authenticating to GCP through Workload



Identity Federation. The same workflow can rebuild an older commit under a new version code for rollbacks.

- **Tested end to end** on an Android emulator against a local backend and local Headscale: silent update, a crashing build rolled back automatically, self-update keeping its IP, process kill and reboot.
- *Representative PRs*: APK-CMS #20 Updater app: a second APK that keeps the Player updated · #25 build the Player and Updater for Android 5 as well as Android 7+ · #33 auto-start lock · CMS-Back #102 Updater backend: check-ins, silent updates and rollbacks · #68 an update is not finished until the screen confirms it · #120/#121 two Android floors · #122 set\_autostart command · #124/#126 minute check-ins and retry after power loss · CMS-Web #74 Updater in the CMS · #88 choose the Android version · #89 auto-start lock · Markai-Updater #4 clock repair · CI and release pipelines

## 15 · Trusted clock, play accounting & delivery

- **A play is billed only when it really happened** — it counts when the ad renders and its turn completes; an ad that will not play is benched and surfaced in the CMS instead of being billed; and the backend refuses plays that arrive faster than a slot can physically run.
- **Trusted clock** — the server sends its time on every heartbeat and the Player runs on it, so proof of play recorded offline is back-dated to the corrected time and a box with a wrong clock cannot produce wrong billing. The CMS shows when a screen last beat on an unverified clock.
- **Play books** — plays are identified by a fingerprint rather than a counter and kept in books; a new book starts when one fills, and the backend can tell a planned new book from a box that lost its counter.
- **Delivery** — exact expected plays, a daily delivery ledger, expected vs delivered on the campaign page, and a proof-of-play sheet that is reviewed (and editable) before it is downloaded. Proof of play is also exposed over the partner API and filterable by campaign or several ads at once.
- *Representative PRs*: APK-CMS #34 bill a play only when the ad renders · #37 count a play only when its turn completes, bench an ad that will not play · #38 trusted clock · #23 a book of plays · CMS-Back #107 identify a play by its fingerprint · #108 planned new book vs lost counter · #116 exact expected plays, daily delivery ledger · #118 review the proof-of-play sheet · #127 refuse plays faster than a slot · #128 trusted clock · #89 proof-of-play over the partner API · CMS-Web #85/#87 delivery · #90 unverified clock · #93 warn when a screen cannot play an ad

## 16 · Player hardening, rotation & live campaign editing

- **Offline resilience** — the Player keeps playing cached ads after a reboot with no network and survives a torn settings file instead of crash-looping on it.
- **Two Android floors** — the Player and Updater build for Android 5 and Android 7+ from one codebase using product flavours, and the CMS picks the right build per box.
- **Creative rotation** — one slot, several creatives, a different one on each loop pass; a milestone photographs the creative it names.
- **Slots and layout** — every slot has its own airtime, ads can be dragged between slots, and an ad is placed on the panel and rendered at the panel's native size.
- **A campaign is edited for its whole life, never replaced** — add an alternate ad, swap it, drop it or reorder on a live campaign, edit the schedule, pause, cancel or delete behind a typed confirmation, with an activity log of every change to the loop.
- *Representative PRs*: APK-CMS #16 creative rotation · #17 a milestone photographs the creative it names · #35 keep playing cached ads after a reboot with no network · #36 survive a torn settings file · CMS-Back #46 every slot its own airtime · #64 rearrange ads between slots · #65 creative rotation ·

#67 edit a live campaign's ads · #75 place an ad and render at native size · #85 a campaign is edited for its whole life · #115 log every change to the loop · CMS-Web #49 drag & drop between slots · #52 manage a live campaign's ads · #59 place an ad on the screen

## 17 · WhatsApp & Gmail ingestion

- **WhatsApp** — an operator links their handset to the co-pilot from the CMS (many numbers per account), books by chat, and gets structured booking slips with one fact per line and warnings kept apart. Milestone proof photos are sent to the booker and to the screen's owner.
- **Gmail** — advertiser emails are turned into bookings, behind a review-and-confirm panel so nothing is booked unseen.
- **Co-pilot rebuilt around what operators actually ask** — reports and screenshots on request, auto-naming, multi-screen picking, fitting a creative from chat, and draft state and consent held by the server rather than the model.
- *Representative PRs:* CMS-Back #44/#45 Gmail ingestion · #48 WhatsApp ingestion · #49 reports, screenshots on request, auto-naming · #50 take draft state and consent out of the model's hands · #53 structured booking slips · #54/#58 milestone shots over WhatsApp, many numbers per account · #73 rebuild the co-pilot around what the operator actually asks · CMS-Web #30 Gmail review-and-confirm panel · #34/#43 WhatsApp link panel · #36 images and downloadable reports in chat

## 18 · The CMS as a phone app (Compose Multiplatform)

- Built `CMS-Mobile`, the operator CMS as a native phone app for **Android and iOS from one Kotlin codebase** (Kotlin Compose Multiplatform).
- Scheduling with a screen filter, booking a campaign, editing and deleting campaigns behind a typed confirmation, placing an ad on the panel, Creative Fit upload, campaign frames and capture, proof-of-play filters and gallery, and delivery review before download.
- Same role rules as the web CMS (player app and updates are superadmin-only), its own design system with searchable dropdowns, and a release pipeline that fails fast on missing secrets or a bad keystore. The backend serves a mobile-app update endpoint and the web CMS has a download page.
- *Representative PRs:* CMS-Mobile — Mark AI CMS mobile app (Android + iOS) · campaign frames + capture, Creative Fit upload flow · edit booked campaigns, place ads on the panel · filters for Scheduling and Proof of play · review the proof of play before downloading · CMS-Back #105 mobile-app update endpoint · CMS-Web #79 mobile app download page

## 19 · Screen hardware health (NovaStar VNNOX)

- Integrated NovaStar's **VNNOX Care** API so the CMS shows the LED panel's real hardware state: cabinet pixel size, module type and dead cabinets, polled every minute with a 15-second guarded manual refresh.
- **Dead cabinets are painted onto proof screenshots** as dark patches, so an advertiser's proof shows what the panel really displayed. Fixed a decimal cabinet index and a timezone error that were painting healthy tiles dark.
- Operators can correct a screen's resolution and book against it; screen health is superadmin-only and grantable per screen owner.
- *Representative PRs:* CMS-Back #91 VNNOX Care screen health, player keyword and dark cabinets on screenshots · #93 correct a screen's resolution · #95/#96 cabinet index and fault-time fixes · #98/#99 health grants · #110 poll every minute · #125 controller time zone · #130 refresh from VNNOX + cabinet



pixel size and module type · CMS-Web #64 screen health pages · #96 real-size cabinets and a Modules view

## 20 · Mark AI Studio & the developer dashboard

- **Creative Fit grew up** — it now accepts video, lets the user say what is wrong with a rebuilt ad and redo it, decides crop vs redraw for the caller, and supports placing the ad by hand.
- **Mark AI Studio** (`studio.mark-ai.tech`) — split Creative Fit out of the CMS into its own public product: a Next.js workbench and a separate FastAPI service with its own users, job service and worker pool, a cancel that really cancels, streaming media helpers so a 100 MB file fits in 1 GiB of RAM, a public API with keys, and its own Cloud Build / Cloud Run pipeline.
- **GCP developer dashboard** (`MarkAi-dev`) — a read-only dashboard that prices every billable resource, breaks cost down per product, and shows live utilisation, workers and spend for a chosen date range.
- *Representative PRs*: CMS-Back #72 correct a rebuilt ad by saying what's wrong · #76 a public Creative Fit API, video stretch and explicit job ownership · #86 remove Studio from cms-back · API-studio — Studio's own job service and worker pool, streaming media helpers, Dockerfile and Cloud Build pipeline · Mark-ai-studio — the public resizer, one button for crop vs redraw, place the ad by hand · MarkAi-dev — read-only GCP developer dashboard, price every billable resource

## 17 Month-by-month timeline

| Period            | What I shipped                                                                                                                                                                                                                                                                                                                                                     |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Jun 2026          | Platform rebuilt from the first commit (15 Jun). Proof-of-play with batching; <b>the WebSocket removed</b> in favour of the heartbeat; deployment pipelines.                                                                                                                                                                                                       |
| Early Jul 2026    | Signed GCS URLs across dashboard, AI tools & proof shots; all time interpreted in IST; editable booking schedules; operator-triggered location re-scan; play-based milestone screenshots and the whole capture-correctness pass; AI date tools; two guardrail layers; pagination.                                                                                  |
| Mid Jul 2026      | Per-screen operating hours across backend, CMS and player; the location/fuzzy-matching engine; direct-to-GCS uploads; honest booking lifecycle; mobile-responsive CMS; ext-API attribution; screenowner parity and enrollment keys; plays-trend chart.                                                                                                             |
| Late Jul 2026     | The advertiser platform (marketing site, auth, booking wizard, live slot map, screenowner console); the four-part booking-approval workflow; operator email identity, password reset & email service; on-demand captures; AI campaign Q&A + PoP gap analysis; Razorpay.                                                                                            |
| Early Aug 2026    | Per-owner pre-keyed APKs, release pipeline & fleet version reporting; enrollment-policy pre-check; Creative Fit end to end; capture retention & downloads; campaign PDF and proof-of-play Excel exports; Ethernet-only screen location; re-register on token expiry.                                                                                               |
| Mid-Late Aug 2026 | Server-side pagination and search on every list; Gmail and WhatsApp ingestion; per-slot airtime and drag-and-drop between slots; creative rotation; editing a live campaign's ads; manual screen location; placing an ad on the panel at native size; the co-pilot rebuilt around what operators ask; Creative Fit corrections and video; Mark AI Studio launched. |
| Early Sep 2026    | Studio split into its own backend and pipeline; partner API for screen detail, booking events, creatives and proof of play; advertiser site redesign and Cloud Run container; GCP developer dashboard; demo fleet seeder; VNNOX Care screen health with dark cabinets on screenshots.                                                                              |

| Period        | What I shipped                                                                                                                                                                                                                                                                               |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mid Sep 2026  | <b>The Updater app</b> across Player, backend and CMS — silent installs, health window and automatic rollback; the CMS phone app for Android and iOS; play books and fingerprinted plays; delivery ledger and reviewable proof-of-play sheet; campaign delete and state filters.             |
| Late Sep 2026 | Player and Updater for Android 5 as well as 7+; auto-start lock; offline playback after reboot and torn-settings recovery; billing a play only when it renders; <b>trusted clock</b> and clock repair; install retry after power loss; live VNNOX refresh with cabinet size and module type. |

## 🎯 What this role is teaching me

- **Owning a whole product surface** rather than a slice of one — backend, device, two front-ends and the infrastructure underneath, with no one else to hand a layer to.
- **Software that touches hardware is unforgiving.** A screen sits in someone else's café: it can lose its token, lose GPS, be on an Ethernet cable, be closed for the night, or be watched by nobody. Most of my best work here is making the system behave sanely when it can't assume anything.
- **Evidence over assertion.** Proof-of-play is money, so "the ad played" has to be provable — hence booking-level attribution, render-callback captures and burned-in location overlays.
- **Deleting things is part of building them** — the WebSocket, the proxied upload endpoint and the Delete button on bookings were all removed on purpose, and the system got better each time.
- **An update is a risk, so it needs a way back.** Shipping a build to boxes nobody can reach only works if a bad one is noticed and undone without a person — which is why the Updater has a health window and an automatic rollback rather than just an install button.